INVERTIS UNIVERSITY, BAREILLY

B.Tech CSE – Third Year

APPLIED DATA STRUCTURES & ALGORITHMS LAB (BCS553)

LAB 5

Assignment 5: Linear Search

Assignment aim: Search values sequentially and count occurrences.

Experiment No. 5.1 — Linear search when key is present

1. Experiment Title: Linear search when key is present

2. Aim: Implement the task specified in the BCS553 lab assignment.

3. Software / Tools Required: Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

4. Theory: Linear search checks elements one by one and reports the matching index or -1.

5. Algorithm / Procedure:
Check each element from left to right; return its index if it matches the key; return -1 if no match is found.
Then compile and execute the Java program.

6. Program:

```
public class Task_01_Linear_search_when_key_is_present {
    static int linearSearch(int[] a, int key) {
        for (int i = 0; i < a.length; i++) {
            if (a[i] == key) return i;
        }
        return -1;
    }
    public static void main(String[] args) {
        int[] a = {45, 67, 80, 52, 90};
        System.out.println(linearSearch(a, 80));
    }
}
```

7. Input: Fixed input values are declared in the program: the arrays and search keys specified in each task.

8. Output:
2

Time / Space Complexity: O(n) worst-case time; O(1) auxiliary space.

9. Result: The program implements the specified task and produces the expected output for the given test data.

10. Viva-Voce Questions:

- What does linear search return when a key is found?
- Why return -1?
- Must the array be sorted?

Experiment No. 5.2 — Linear search when key is absent

1. Experiment Title: Linear search when key is absent

2. Aim: Implement the task specified in the BCS553 lab assignment.

3. Software / Tools Required: Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

4. Theory: Linear search checks elements one by one and reports the matching index or -1.

5. Algorithm / Procedure:
Scan each element and compare it with key; because no value matches 100, return -1 after the loop. Then compile and execute the Java program.

6. Program:

```
public class Task_02_Linear_search_when_key_is_absent {
    static int linearSearch(int[] a, int key) {
        for (int i = 0; i < a.length; i++) {
            if (a[i] == key) return i;
        }
        return -1;
    }
    public static void main(String[] args) {
        int[] a = {45, 67, 80, 52, 90};
        System.out.println(linearSearch(a, 100));
    }
}
```

7. Input: Fixed input values are declared in the program: the arrays and search keys specified in each task.

8. Output:
-1

Time / Space Complexity: O(n) worst-case time; O(1) auxiliary space.

9. Result: The program implements the specified task and produces the expected output for the given test data.

10. Viva-Voce Questions:

- What does -1 indicate?
- What is the worst case for linear search?
- Can linear search handle unsorted arrays?

Experiment No. 5.3 — Count occurrences of 52

1. Experiment Title: Count occurrences of 52

2. Aim: Implement the task specified in the BCS553 lab assignment.

3. Software / Tools Required: Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

4. Theory: Linear search checks elements one by one and reports the matching index or -1.

5. Algorithm / Procedure:
Initialize count to zero; traverse all elements; increment for every occurrence of 52; print the count. Then compile and execute the Java program.

6. Program:

```
public class Task_03_Count_occurrences_of_52{
    public static void main(String[] args) {
        int[] a = {52, 10, 52, 7, 52};
        int count = 0;
        for (int value : a) if (value == 52) count++;
        System.out.println("Count: " + count);
    }
}
```

7. Input: Fixed input values are declared in the program: the arrays and search keys specified in each task.

8. Output:
Count: 3

Time / Space Complexity: O(n) time; O(1) auxiliary space.

9. Result: The program implements the specified task and produces the expected output for the given test data.

10. Viva-Voce Questions:

- What is an occurrence?
- Why not stop at the first match?
- Which condition checks equality?